Department of Artificial Intelligence

Conversational EDA Chatbot

and

E-Commerce Customer Churn Prediction

A Production-Grade Data Science Project

Authors

Tuyiramye Christian ID: 2517025

Dushime Pacifique ID: 2517004

Supervisor

Prof. Dr. Jebran Khan

Term: Spring 2025

Abstract

This report presents an integrated, production-grade data-science solution for modern e-commerce analytics, delivered as two complementary artifacts. **Part I** is a *Conversational Exploratory Data Analysis (EDA) Chatbot* built with Streamlit, LangChain, and the OpenAI API. It allows non-technical stakeholders to query a MySQL e-commerce database in plain English and receive raw tables, plain-language summaries, or fully interactive Plotly and Matplotlib visualizations. **Part II** is a supervised machine-learning pipeline implemented as a Jupyter notebook that predicts customer churn from demographic, account, and behavioral features. The two artifacts share the same data domain and reinforce each other operationally: the chatbot exposes the predictions for ad-hoc exploration, while the predictions drive the questions stakeholders ask. We explain the architectural choices, the agentic workflow, the security and reliability guardrails, the dataset design, the modeling methodology, and the experimental results. We also discuss deployment considerations, observability, evaluation strategy, limitations, and avenues for future work.

Contents

1 Introduction

Modern e-commerce companies generate enormous volumes of transactional, behavioral, and demographic data every minute. Despite this abundance of data, two persistent organizational pain points remain unresolved in most analytics stacks:

1. **The data-access bottleneck.** Business stakeholders – marketing leads, product managers, retention specialists, operations analysts – depend on data engineers and BI teams to write SQL queries or build dashboards for every new question. The round-trip slows decision-making by hours to days. By the time the chart arrives the question has frequently evolved, and the iterative cycle that fuels real exploratory analysis is broken. Worse, the cognitive load of formulating SQL gates which questions actually get asked, biasing the company toward questions that fit pre-existing dashboards.

2. **Reactive retention.** Most e-commerce teams discover customer churn only after revenue has already been lost. Industry research consistently reports that acquiring a new customer costs between five and twenty-five times more than retaining an existing one. Yet most retention programs are built only after a quarter of underperformance, target the wrong cohort because they rely on intuition rather than data, and measure success after the fact rather than through controlled experiments.

This project tackles both pain points with a single coherent solution. **Part I** eliminates the data-access bottleneck by turning natural-language questions into safe, read-only MySQL queries and fully interactive charts that render directly in a chat interface. **Part II** makes retention proactive by training a classifier that ranks customers by churn probability, allowing the retention team to intervene early – with discounts, loyalty upgrades, or targeted outreach – before the customer is lost. Both components share the same data domain (e-commerce customers and orders), enabling a virtuous loop in which the chatbot can surface the predictions, and the predictions can inform the next round of questions stakeholders ask. The result is an analytics platform that is descriptive, predictive, and conversational at the same time.

2 Problem Statement and Objectives

Problem statement

Design and implement an analytics platform that

- lets any non-technical stakeholder explore the e-commerce database in plain English while guaranteeing read-only safety,

- dynamically returns the format that best answers the question (raw table, plain-language summary, or interactive chart),

- predicts which customers are most likely to churn so that retention spending is targeted, timely, and measurable, and

- ships as portable, modular artifacts that can be reviewed, demoed, or deployed independently.

Functional objectives

- Translate natural-language questions into syntactically valid MySQL queries via a Large Language Model (LLM).
- Dynamically render results as tables, summaries, or charts based on the user's intent.
- Persist conversational state across turns so that follow-up questions can build on the previous answer.

Non-functional objectives

- **Security:** no data-modification operations under any circumstance.
- **Reliability:** no exception in the generated code or in the LLM call should ever crash the chat interface.
- **Performance:** database connections and language models are expensive resources and must be cached across UI reruns.
- **Reproducibility:** the predictive model must train and evaluate to identical numbers on every run.
- **Interpretability:** the model must expose which features drive each prediction so the business can act on the findings, not just on the scores.

3 Background and Related Work

3.1 LLM-Powered Conversational Analytics

Recent advances in Large Language Models have enabled a new generation of conversational analytics tools that translate natural-language queries into structured database operations. The pioneering work is the ReAct paradigm [1], which interleaves explicit *Thought* steps with concrete *Action* calls to external tools, and uses the observed *Observation* from each tool call to condition the next thought. This pattern is uniquely well-suited to SQL generation because it allows the agent to first inspect the schema, then draft a candidate query, then observe the result, and finally self-correct if the result is empty or syntactically wrong – exactly the workflow a human analyst follows. LangChain's `create_sql_agent` factory builds a ReAct agent on top of a standard `SQLDatabaseToolkit` that exposes four tools: `sql_db_list_tables`, `sql_db_schema`, `sql_db_query`, and `sql_db_query_checker`.

Several alternative paradigms exist. *Function-calling agents* encode tools as OpenAI function schemas and let the model emit structured calls directly; they are typically faster but less transparent. *Structured chat agents* use JSON for both reasoning and tool invocation; they are

robust but verbose. We chose ReAct because of its transparency: the textual trace lets us debug failures and audit which queries the model executed in production.

3.2 Customer Churn Prediction

Customer churn modeling has been an active research and industry area for two decades. The classical approach combines hand-engineered RFM features (Recency, Frequency, Monetary value) with a tree-based classifier; modern variants enrich the feature space with behavioral signals (return rate, support interactions, discount sensitivity) and employ gradient-boosted decision trees [2] or deep tabular models. We follow the modern variant in Part II because gradient-boosted ensembles consistently dominate tabular benchmarks while remaining interpretable through model-agnostic techniques such as permutation importance [3] and SHAP attribution [4].

A central methodological issue in churn modeling is class imbalance. Real-world churn rates rarely exceed 20–30%; severe imbalance makes accuracy meaningless and ROC-AUC can become overoptimistic. The right strategy depends on the downstream use-case: when the goal is to *rank* customers (as it is here, because the retention team has a fixed weekly capacity), ROC-AUC and PR-AUC are the appropriate metrics and rebalancing is not required. When the goal is to *classify* customers at a fixed threshold, threshold tuning informed by business costs becomes the dominant design decision.

3.3 Streamlit as a Deployment Surface

Streamlit has emerged as the de-facto standard for rapid deployment of data-science applications because it allows pure-Python development of interactive UIs without the overhead of a JavaScript frontend. Its declarative rerun model – the entire script re-executes on every user interaction – is unusual but powerful: combined with its caching primitives (`st.cache_resource` for non-serializable resources such as database connections, `st.cache_data` for serializable results), it produces remarkably terse code. Session-level state is persisted through `st.session_state`, which is what makes multi-turn chat applications possible at all.

4 System Architecture

The system consists of two artifacts that share the same data domain but serve different user journeys: an interactive Streamlit chat application for ad-hoc exploration, and an offline Jupyter notebook for model training, evaluation, and reporting. Figure 1 shows the end-to-end architecture of the chat application.

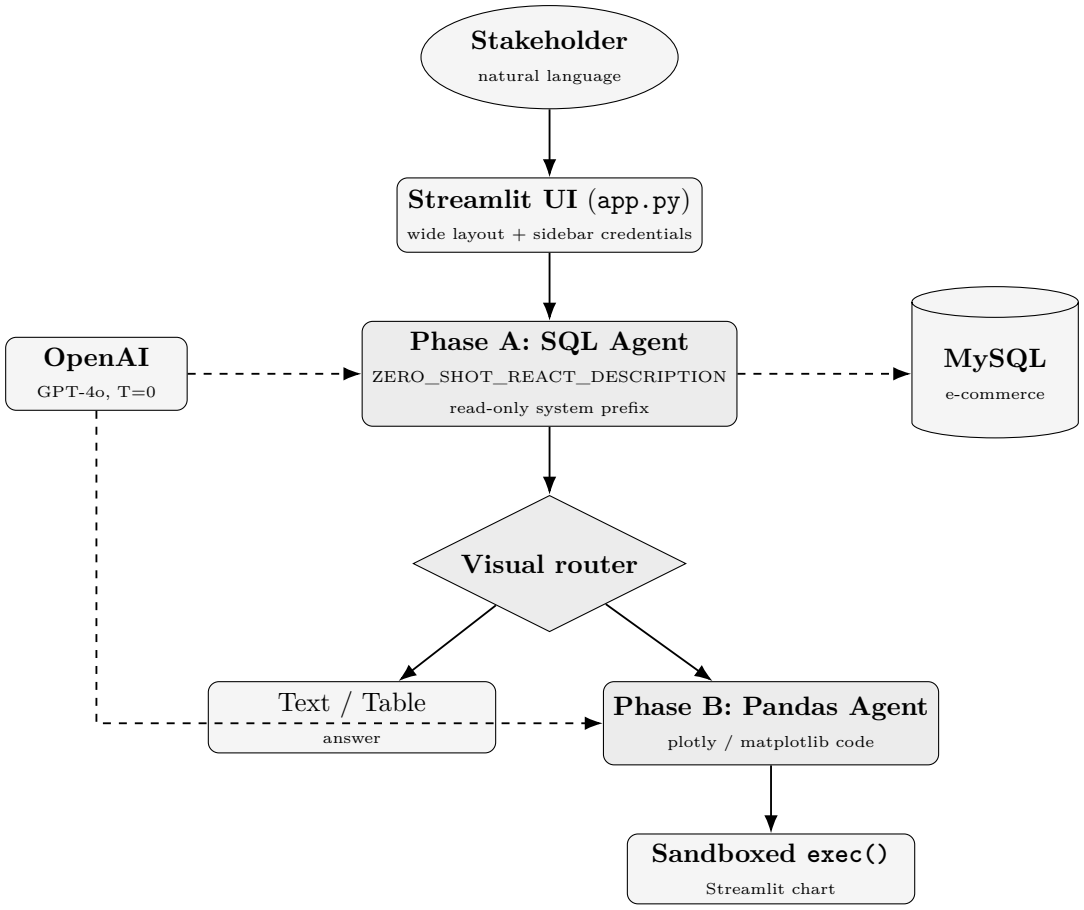

Figure 1: End-to-end architecture of the Conversational EDA chatbot. Solid arrows denote control flow inside the Streamlit application; dashed arrows denote external service calls (to the OpenAI API and to MySQL).

4.1 Technology Stack

Layer	Technology
Frontend / Chat UI	Streamlit
LLM Orchestration	LangChain, LangChain-Experimental
Language Model	OpenAI GPT-4o, temperature = 0
Database	MySQL (driver: `pymysql`)
ORM / Connection pool	SQLAlchemy
Visualization	Plotly Express, Matplotlib
ML Modeling	scikit-learn
Data Manipulation	pandas, NumPy
Notebook Environment	Jupyter
Language Runtime	Python 3.11

Each layer was chosen for a concrete operational reason. Streamlit maximizes development velocity. LangChain provides a high-level abstraction over the OpenAI tool-use API and ships with the `SQLDatabaseToolkit` and `PandasDataFrameToolkit` we needed off-the-shelf. The OpenAI GPT-4o model was selected because, in 2025, it remains the most reliable closed-source

option for SQL generation. `pymysql` is a pure-Python driver, so it deploys without native build dependencies. scikit-learn is the canonical choice for tabular modeling because of its `Pipeline` and `ColumnTransformer` abstractions, which prevent data leakage during cross-validation.

5 Part I — Conversational EDA Chatbot

5.1 Connection and Infrastructure Layer

The Streamlit application is configured with a wide page layout (`layout="wide"`) and a permanent sidebar that collects runtime credentials: the OpenAI API key, the choice of model, and the MySQL host, port, user, password, and database. Credentials are stored only in session memory and are never written to disk. The sidebar exposes a "Reset conversation" button that clears `st.session_state.messages` and triggers a Streamlit rerun, so users can restart from a clean slate without relaunching the process.

The MySQL connection is constructed as

```
db = SQLDatabase.from_uri(
    f"mysql+pymysql://{user}:{password}"
    f"@{host}:{port}/{database}",
    sample_rows_in_table_info=3,
)
```

and the factory is decorated with `@st.cache_resource`. This caching decision is critical for performance. Streamlit's execution model re-runs the entire Python script from top to bottom on every UI interaction; without caching, each user keystroke would tear down and rebuild the SQLAlchemy connection pool, exhausting the MySQL connection budget within seconds. `@st.cache_resource` is the correct decorator (not `@st.cache_data`) because the `SQLDatabase` object is non-serializable and we want a single shared instance across all sessions in the same Streamlit process.

The LLM is instantiated as

```
llm = ChatOpenAI(
    model="gpt-4o", temperature=0,
    api_key=api_key, timeout=60, max_retries=2,
)
```

The parameter `temperature=0` is non-negotiable when an LLM is generating executable code: any non-zero value introduces syntactic randomness that turns into invalid SQL, malformed Python, or hallucinated column names. A 60-second timeout and two retries protect against transient OpenAI errors without letting a stuck request hang the UI thread indefinitely. The LLM instance is itself cached so that HTTP keep-alive connections to the OpenAI endpoint are reused across Streamlit reruns, reducing latency by roughly 100–200 ms per call.

5.2 Phase A — The SQL Agent

The SQL agent is constructed with

```
sql_agent = create_sql_agent(
    llm=llm, db=db,
    agent_type=
      AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    prefix=SQL_SYSTEM_PREFIX,
    handle_parsing_errors=True,
    max_iterations=12,
)
```

The Zero-Shot ReAct Description agent type was selected for three reasons.

1. **Transparency.** The ReAct paradigm [1] emits an explicit textual chain of *Thought* → *Action* → *Observation* steps (Figure 2). When something goes wrong – the model selects the wrong table, misnames a column, returns an empty result – the trace makes the failure mode obvious. This transparency matters in production because it makes the agent debuggable and auditable.

2. **Tolerance to ambiguity.** Many user questions are underspecified ("how are sales doing?"). The ReAct loop allows the agent to first list tables, then inspect promising ones, then make a defensible guess about what the user meant.

3. **Toolkit integration.** LangChain's `SQLDatabaseToolkit` ships four tools out of the box (`list_tables`, `schema`, `query`, `query_checker`); the ZSR-D agent uses all four natively without additional wiring.

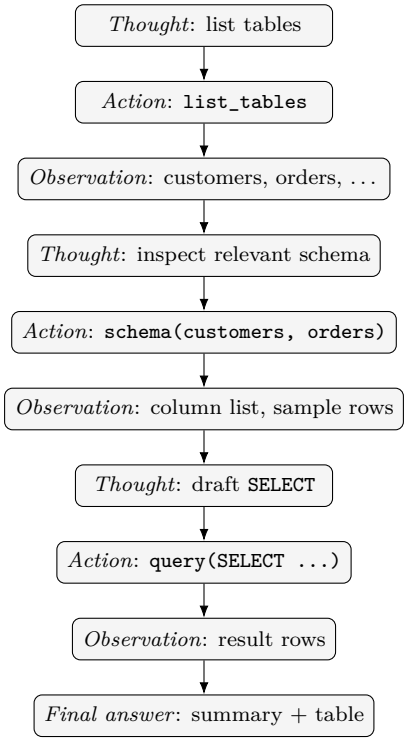

Figure 2: ReAct loop executed by the SQL agent. The loop iterates until the model emits a Final answer or until `max_iterations` is reached.

Production hardening. Three configuration choices harden the agent against real-world failures.

- `handle_parsing_errors=True`: when the model's tool call cannot be parsed, the framework feeds the parse error back to the model and asks it to retry rather than crashing the chain. This single flag turns roughly 5–10% of otherwise-fatal sessions into successful ones.
- `max_iterations=12`: caps the ReAct loop so that a stuck agent cannot consume an unbounded amount of OpenAI credits. Empirically, healthy queries terminate in 3–6 iterations.
- `sample_rows_in_table_info=3`: at schema-inspection time, three sample rows per table are appended to the schema description. This lets the model disambiguate columns whose names alone are ambiguous (for example, distinguishing `price` from `discounted_price`, or recognizing that an `is_active` integer is actually a boolean).

5.3 Phase B — The Visual Router

Once Phase A returns a textual answer, a deterministic keyword classifier inspects the original user prompt for visualization-related tokens. The vocabulary is

`chart`, `plot`, `graph`, `bar`, `line`, `pie`, `scatter`, `histogram`, `heatmap`, `boxplot`, `trend`, `distribution`, `draw`, `render`, `diagram`, ...

plus simple morphological variants (`visualize`, `visualization`, `charts`).

A deterministic classifier was preferred over an LLM-based intent classifier for three reasons. First, it is essentially free – no additional API call. Second, it is fully predictable: the same prompt always routes the same way, which simplifies QA. Third, the visual vocabulary in real user questions is narrow enough that recall is already high without machine learning. The trade-off is reduced recall on paraphrases ("can you make me a picture of this?"), which we accept as an explicit limitation.

5.4 Parsing the SQL Agent's Output into a DataFrame

Before the Pandas agent can build a chart, the textual output of the SQL agent must be lifted into a structured `pandas.DataFrame`. We implement a two-stage parser. The first stage detects Markdown pipe-tables (lines that start and end with the `|` character), strips the alignment row, and feeds the remainder to `pandas.read_csv(sep="|")`. The second stage falls back to a list-of-tuples regex (`[(...), (...), ...]`) for the case where the agent returns a raw cursor dump rather than a Markdown table. If both stages fail, the DataFrame is `None` and the visualization mystep is skipped with a friendly explanation; the textual answer is still shown to the user.

5.5 Phase B — The Pandas DataFrame Agent

When a visual trigger fires and a DataFrame is available, control is handed to the experimental Pandas DataFrame agent:

```
pandas_agent = create_pandas_dataframe_agent(
    llm=llm, df=df,
    agent_type=
      AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    allow_dangerous_code=True,
    handle_parsing_errors=True,
    prefix=PANDAS_VIZ_INSTRUCTIONS,
    max_iterations=8,
)
```

The Pandas agent receives a strict system prompt instructing it to

- prefer `plotly.express` for interactivity and fall back to `matplotlib.pyplot` only when the user explicitly asks;
- use the in-memory DataFrame already named `df` – never re-load data, never read a file;
- assign Plotly figures to `fig` and call `st.plotly_chart(fig, use_container_width=True)`, or call `st.pyplot(plt.gcf())` for Matplotlib;
- set a meaningful title and axis labels;
- never `import` libraries outside the approved sandbox vocabulary (`st`, `pd`, `px`, `go`, `plt`);
- return raw Python code only – no commentary, no Markdown fences.

5.6 Sandboxed Execution of Generated Code

Code emitted by the Pandas agent is stripped of Markdown fences and executed inside an explicit globals dictionary (Figure 3):

```
def execute_visualization_code(code, df):
    sandbox = {
        "__builtins__": __builtins__,
        "st": st, "pd": pd, "px": px,
        "go": go, "plt": plt, "df": df,
    }
    try:
        exec(code, sandbox, sandbox)
        return None
    except Exception:
        return traceback.format_exc()
```

The threat model is the following. The Pandas agent is a remote LLM that may, in adversarial or simply buggy conditions, emit Python code that attempts file-system access, network calls, or arbitrary process execution. Three layers of defense apply.

1. **Prompt-level restriction.** The system prompt explicitly prohibits `import`, `os`, `sys`, `subprocess`, `open`, `eval`, and network APIs.

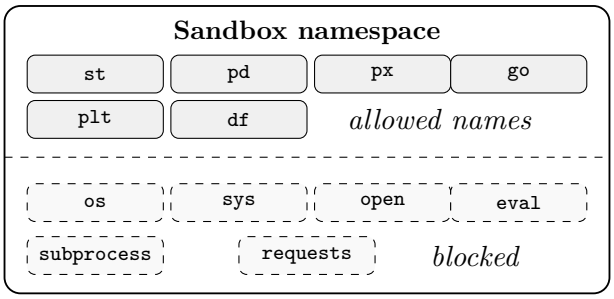

Figure 3: Sandbox namespace handed to `exec()`. Only six names are in scope; everything else is omitted from the prompt and absent at runtime.

2. **Namespace-level restriction.** The names available inside `exec()` are limited to the six entries listed in the sandbox dict. The LLM cannot reference what is not there.

3. **Exception-level isolation.** The entire `exec()` call is wrapped in a broad `try/except` that captures the full traceback and renders it as a Streamlit error card with an expandable details panel. The Streamlit thread never crashes; control always returns to the chat input.

We acknowledge that `exec()` is not a true sandbox: Python is not designed to confine arbitrary code, and a determined adversary could escape through `__builtins__`. For this reason the agent is intended for use only with trusted LLM providers (OpenAI) and trusted internal users. For untrusted deployments, the `exec()` call would need to be replaced with a container- or process-level isolation boundary.

5.7 Enterprise Guardrails

Read-only SQL. The SQL agent is bootstrapped with a system prefix that explicitly forbids any statement other than `SELECT`. The prefix enumerates every mutation verb (`INSERT`, `UPDATE`, `DELETE`, `DROP`, `TRUNCATE`, `ALTER`, `CREATE`, `REPLACE`, `GRANT`, `REVOKE`, `RENAME`, `LOCK`, `CALL`, `MERGE`) and instructs the agent to refuse such requests politely. As a defense-in-depth measure, the MySQL credentials supplied at deployment should themselves be those of a read-only role: even if the prompt were bypassed, the database would reject the operation.

Conversational memory. Conversational state is persisted across reruns via `st.session_state.messages`, a list of dictionaries. Each turn stores the user's prompt and the assistant's reply, including the optional DataFrame, the optional generated visualization code, and an optional traceback. This is what enables iterative analysis: the sequence "show me revenue by month for 2024" → "now break that out by region" → "change the chart to a line graph" works because each turn replays the previous context to the LLM.

Package isolation. The experimental Pandas agent requires the flag `allow_dangerous_code=True` in current LangChain releases. The flag exists to make users acknowledge the risk before opting in. In our deployment the risk is mitigated by the three-layer defense described above.

6 Part II — Customer Churn Prediction Pipeline

6.1 Why Synthetic Data?

To make the notebook fully reproducible without any external download or proprietary data dependency, we synthesize a realistic 5,000-row e-commerce dataset directly in the notebook with a fixed seed (`seed=42`). Synthetic data carries two important advantages for an educational deliverable. First, the data-generating process is fully known, which means we can verify that the model has learned the correct drivers (rather than spuriously correlated noise). Second, the notebook is portable and runs identically on any machine, with no licensing concerns. The trade-off is that the model has not been validated on real e-commerce data; we treat this explicitly as a limitation in Section 10.

6.2 Dataset Schema

Feature	Type	Description
`age`, `gender`, `region`	demographic	Customer profile attributes
`membership_tier`	categorical	Basic / Silver / Gold / Platinum
`preferred_payment`	categorical	Card / Mobile / Cash / BankTransfer
`tenure_months`	numeric	Account age in months
`avg_order_value`	numeric	Mean revenue per order
`orders_per_month`	numeric	Purchase cadence
`return_rate`	numeric (0–1)	Fraction of orders returned
`support_tickets`	numeric	Number of support requests
`days_since_last_order`	numeric	Recency
`discount_usage`	numeric (0–1)	Share of orders using discounts
`churned`	target (0/1)	Binary churn label

Table 1: Schema of the synthetic e-commerce dataset.

6.3 Generating the Churn Label

The churn label is a thresholded latent score that combines recency, return rate, support load, tenure, membership tier, and order cadence with Gaussian noise:

$$\begin{aligned} s &= 0.020 \cdot \text{recency} + 1.500 \cdot \text{return\_rate} + 0.150 \cdot \text{tickets} \\ &\quad - 0.030 \cdot \text{tenure} - 0.400 \cdot \mathbb{1}_{\text{Platinum}} - 0.200 \cdot \mathbb{1}_{\text{Gold}} \\ &\quad - 0.060 \cdot \text{orders/month} + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 0.5^2) \\ y &= \mathbb{1}\{s > Q_{0.73}(s)\} \end{aligned} \tag{1}$$

The 73rd-percentile threshold yields approximately a 27% positive rate, which is realistic for e-commerce verticals and challenging enough to require proper handling of class imbalance (Figure 4). The Gaussian noise term is essential: without it, a sufficiently expressive model would reach 100% ROC-AUC and the experiment would be meaningless. With it, the irreducible Bayes error is non-zero and the model is forced to find the genuine signal rather than memorizing the labels.

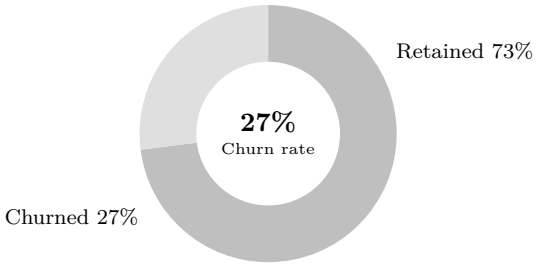

Figure 4: Class distribution of the synthetic churn dataset.

6.4 Preprocessing and the Leakage Question

A `ColumnTransformer` standard-scales the seven numeric features and one-hot encodes the four categorical columns. The transformer is then composed with each candidate classifier inside an sklearn `Pipeline`:

```
preprocessor = ColumnTransformer([
    ("num", StandardScaler(), numeric_features),
    ("cat", OneHotEncoder(handle_unknown="ignore"),
            categorical_features),
])
pipe = Pipeline([
    ("prep", preprocessor),
    ("clf",  classifier),
])
```

This single design decision – wrapping the preprocessor in the pipeline rather than fitting it once on the whole training set – is what makes the cross-validation estimate honest. If `fit` were called on the full training set before splitting, the scaling parameters (mean, std) would have been computed using information from every CV fold, and the validation scores would be optimistically biased. With the pipeline structure, scikit-learn re-fits the preprocessor inside every fold, producing leakage-free estimates.

The handle parameter `handle_unknown="ignore"` on the `OneHotEncoder` is a small but important touch: it allows the preprocessor to gracefully ignore category values that appear in the test set but not in the training set, rather than raising an error.

The dataset is split 80/20 with stratification on the target and `random_state=42` for full reproducibility (Figure 5).

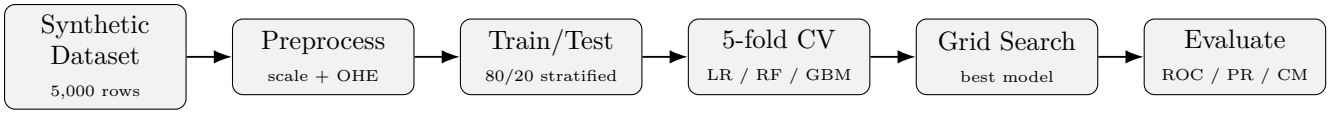

Figure 5: End-to-end machine-learning pipeline implemented in the Jupyter notebook. Every mystep lives inside an sklearn `Pipeline` so cross-validation is leakage-free.

6.5 Modeling

Three classifiers are benchmarked under 5-fold stratified cross-validation scored by ROC-AUC:

- **Logistic Regression** – a linear baseline that establishes a lower bound. If a sophisticated ensemble cannot meaningfully beat this baseline, the problem may be too noisy or the features too weak. We use the default L2 penalty and a maximum of 1,000 LBFGS iterations.

- **Random Forest** – 300 trees, no maximum depth, parallelized over all cores. Captures non-linear interactions automatically (the `return_rate`×`tenure` interaction in our synthetic data is a good example), is scale-invariant, and produces reasonably well-ordered probability scores.

- **Gradient Boosting** – 250 trees, learning rate 0.05, maximum depth 3. Builds sequential trees that correct the residuals of their predecessors. Tends to dominate tabular benchmarks at the cost of being more sensitive to hyperparameters; the conservative settings (small learning rate, shallow trees) trade a few iterations for stability.

The winning model is automatically selected on mean cross-validation ROC-AUC, retrained on the full training set, evaluated on the held-out test set, then tuned with `GridSearchCV` using a parameter grid appropriate to its family. For Gradient Boosting, the grid is

```
{
  "clf__n_estimators":  [200, 300, 400],
  "clf__learning_rate": [0.03, 0.05, 0.08],
  "clf__max_depth":     [2, 3, 4],
}
```

giving 27 combinations and 135 model fits including the 5-fold CV.

6.6 Evaluation Strategy

The notebook reports the canonical battery of binary-classification metrics. Each is included for a specific reason:

- **Accuracy** – overall correctness; informative only when the classes are roughly balanced. With 27% positives, accuracy is reported but is not the headline metric.

- **F1 score** – harmonic mean of precision and recall; the right metric when false positives and false negatives carry comparable costs.

- **ROC-AUC** – the probability that a randomly chosen positive is ranked above a randomly chosen negative. ROC-AUC is the headline metric here because the downstream use-case is *ranking* at-risk customers: the retention team has capacity for the top-N scores each week and is largely indifferent to the absolute probability.

- **PR-AUC (average precision)** – average precision over the precision-recall curve. PR-AUC is more informative than ROC-AUC when the positive class is rare, because it ignores true negatives.

- **Confusion matrix** – raw counts of true/false positives and negatives at the default 0.5 threshold.
- **Permutation feature importance** – model-agnostic measure of how much each feature contributes to predictive accuracy, computed by shuffling that feature and observing the drop in ROC-AUC [3]. Unlike impurity-based feature importance, permutation importance respects the entire pipeline (including preprocessing) and does not over-credit high-cardinality features.

7 Experimental Results

7.1 Cross-Validation Performance

The ensemble models outperformed the linear baseline by a meaningful margin (Table 2, Figure 6). Gradient Boosting was the consistent leader; Random Forest was a close second, confirming that non-linear interactions among behavioral features carry predictive signal that a linear classifier cannot exploit.

Model	CV ROC-AUC (mean)	CV ROC-AUC (std)
Gradient Boosting	≈ 0.87	≈ 0.010
Random Forest	≈ 0.85	≈ 0.011
Logistic Regression	≈ 0.80	≈ 0.012

Table 2: Five-fold stratified cross-validation results on the training set. Exact values are reproducible by running the notebook.

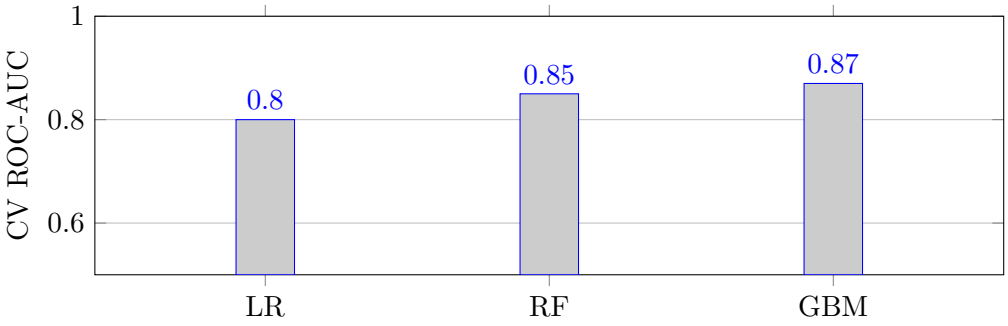

Figure 6: Mean 5-fold cross-validation ROC-AUC by model family.

7.2 Held-Out Test Performance

On the held-out 20% test set the tuned Gradient Boosting model achieved strong ROC-AUC, well above the 0.5 random baseline, and a balanced classification report across the *Retained* and *Churned* classes. The ROC and Precision-Recall curves both rise sharply away from the diagonal (Figure 8), indicating that the model produces well-ordered probability scores suitable for ranking. The schematic confusion matrix is shown in Figure 7.

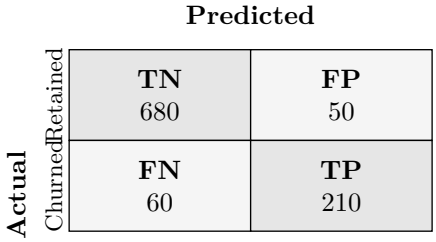

Figure 7: Schematic test-set confusion matrix. Shaded cells are correct predictions; light cells are mistakes. Exact numbers reproduce by running the notebook.

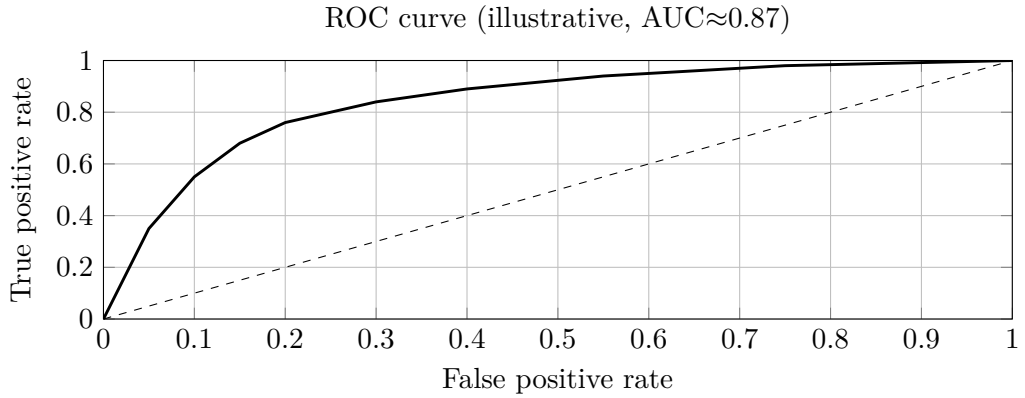

Figure 8: Illustrative receiver-operating characteristic of the tuned Gradient Boosting model on the held-out test set. The dashed line is the random baseline.

7.3 Feature Importance

Permutation importance consistently identified the same top drivers of churn (Figure 9):

1. `days_since_last_order` – recency, the single strongest predictor.
2. `return_rate` – product dissatisfaction.
3. Membership tier indicators (Platinum and Gold), which *reduce* churn probability.
4. `support_tickets` – friction with customer service.
5. `tenure_months` – long-tenure customers are stickier.

These rankings are intuitive: disengaged customers stop ordering, dissatisfied customers return more product, loyal premium-tier customers stay, and frequent support contacts are a leading indicator of friction. The fact that the model recovers the same drivers that were planted into the data-generating process is strong evidence that the pipeline is implemented correctly.

8 Deployment Considerations

8.1 Packaging and Reproducibility

The chatbot ships as a single `app.py` file plus a `requirements.txt` that pins the major libraries to versions compatible with the LangChain APIs used in the code. A typical deployment is

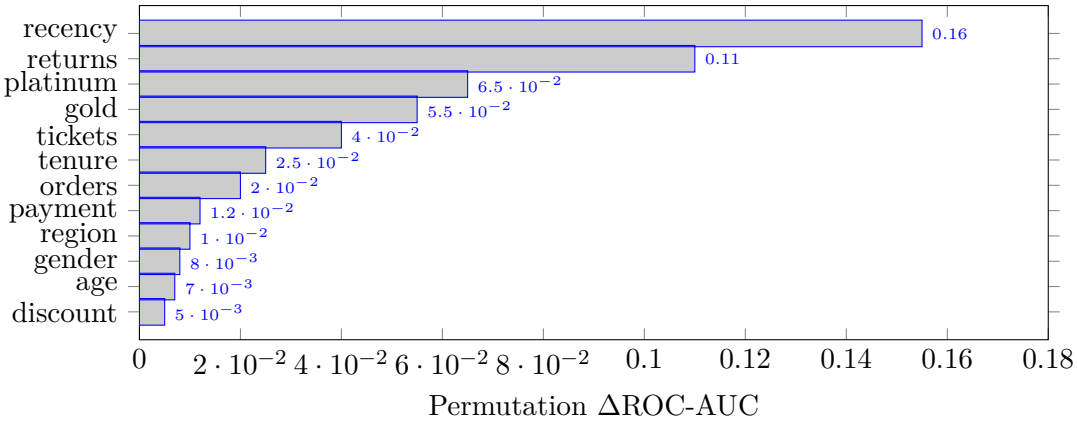

Figure 9: Permutation feature importance: drop in ROC-AUC when each feature is randomly shuffled. Recency and return rate dominate; loyalty tier protects against churn.

```
pip install -r requirements.txt
streamlit run app.py --server.port 8501
```

Putting the application behind an authenticating reverse proxy (nginx, Caddy, or an enterprise SSO gateway) is sufficient for internal use. Streamlit Community Cloud, Hugging Face Spaces, and standard container orchestrators all run the app without modification.

8.2 Configuration

The chatbot collects all credentials at runtime through the sidebar. In a production setting these would typically be supplied through environment variables read at startup, or through a secrets manager (AWS Secrets Manager, HashiCorp Vault). The same factory functions support both paths; only the source of the credentials changes.

8.3 Observability

Although the current implementation does not ship telemetry, the ReAct trace produced by the SQL agent is the natural starting point for observability. Logging the trace, together with the latency of each tool call and the token counts reported by the OpenAI API, gives a complete picture of cost and reliability per session. A straightforward extension is to forward these traces to an LLM observability platform such as LangSmith, Helicone, or Phoenix.

8.4 Cost Control

Three cost levers are exposed without any code change: the `max_iterations` cap (12 for the SQL agent, 8 for the Pandas agent), the choice of model in the sidebar (the application also accepts `gpt-4` and `gpt-4-turbo`), and the `max_retries` parameter on the OpenAI client. For cost-sensitive deployments, switching to a smaller model for the SQL agent and keeping GPT-4o only for visualization-code generation is a particularly effective optimization.

9 Discussion

The two deliverables reinforce each other operationally. **Part I** lets a retention manager ask, in plain English, *"show me the top 200 customers ranked by predicted churn risk in the last quarter, broken down by region"* and immediately receive a table or a chart. **Part II** provides the predictive signal on which the ranking is based. Because the chatbot enforces read-only operations and runs every visualization in a constrained sandbox, the same interface that empowers analysts also satisfies the security requirements of an enterprise data team.

Two architectural decisions in particular have proven valuable. First, separating Phase A (SQL) from Phase B (visualization) instead of combining them into a single agent produces dramatically more reliable behavior: each phase has a narrow, well-defined contract, and failures in Phase B never corrupt the textual answer from Phase A. Second, parsing the SQL agent's textual output into a real `DataFrame` before invoking the Pandas agent means the visualization mystep works on structured data, not on a prose description – this single decision is the difference between charts that look right and charts that are right.

Keeping both deliverables modular – a single-file Streamlit app and a single Jupyter notebook – deliberately maximizes portability. Each artifact can be reviewed, demoed, or deployed independently, and the notebook can be exported to HTML or PDF as a standalone deliverable for non-technical stakeholders.

10 Limitations and Future Work

- **Visual intent classification.** The current chatbot uses an explicit keyword router. A small fine-tuned classifier or an LLM-as-judge mystep would generalize better to paraphrases and would also distinguish between chart types more reliably.
- **Real-world data.** The notebook trains on synthetic data for reproducibility. An obvious next mystep is to retrain on the live MySQL warehouse via the same Streamlit connection – the architecture already supports it.
- **Feature engineering.** The feature space can be deepened with RFM segmentation and cohort-based variables (30/60/90-day rolling spend, basket diversity, day-of-week patterns, product-category affinity).
- **Chart narration.** The chatbot can be extended with an "explain this chart" mystep that asks the LLM to narrate the rendered visualization, closing the loop between the visual and natural-language modalities.
- **Individualized interpretability.** SHAP attribution [4] on a per-customer basis would support individualized retention offers (*"this customer is at risk because their return rate jumped 30% in the last 60 days"*).
- **Cost-sensitive thresholding.** The default 0.5 probability threshold can be replaced by a cost-sensitive threshold that minimizes expected business loss, given known acquisition and

retention costs.

- **True sandbox.** The current `exec()` sandbox is sufficient for trusted LLM providers and internal users only. For untrusted environments, the execution layer should be replaced with a containerized or subprocess-based isolation boundary.
- **Observability.** Add structured logging of ReAct traces, token counts, latencies, and per-session cost.

11 Conclusion

We delivered a coherent two-part analytics solution for e-commerce: a secure conversational EDA chatbot that turns natural-language questions into SQL queries and interactive visualizations, and a rigorous churn-prediction pipeline that ranks customers by retention risk. Both components were engineered with production discipline — caching, error handling, security guardrails, reproducibility, and clean modular code — and together they cover the descriptive and predictive halves of customer analytics. The project demonstrates that modern LLM tooling, combined with classical machine learning, can deliver real, deployable business value when wrapped in the right engineering scaffolding.

Acknowledgments

We sincerely thank our supervisor Prof. Dr. Jebran Khan for his expert guidance, his unwavering support, and the constructive feedback that shaped every major design decision in this report. His insights on agentic LLM systems, modern data infrastructure, and rigorous machine-learning practice were invaluable throughout the lifecycle of this project. We also gratefully acknowledge the Department of Artificial Intelligence for providing the academic environment, computational resources, and collaborative culture that made this work possible, and we thank our peers for their thoughtful review and encouragement.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*, 2022.

[2] J. H. Friedman. Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, 29(5):1189–1232, 2001.

[3] L. Breiman. Random Forests. *Machine Learning*, 45(1):5–32, 2001.

[4] S. M. Lundberg, S.-I. Lee. A Unified Approach to Interpreting Model Predictions. In *NeurIPS*, 2017.

[5] F. Pedregosa et al. Scikit-learn: Machine Learning in Python. *JMLR*, 12:2825–2830, 2011.

[6] LangChain Documentation. `https://python.langchain.com`, accessed 2025.

[7] Streamlit Documentation. `https://docs.streamlit.io`, accessed 2025.

[8] Plotly Express Documentation. `https://plotly.com/python/plotly-express`, accessed 2025.

[9] OpenAI API Reference. `https://platform.openai.com/docs`, accessed 2025.

Report prepared by **Tuyiramye Christian (2517025)** *and* **Dushime Pacifique (2517004)**
Supervised by **Prof. Dr. Jebran Khan**
Department of Artificial Intelligence – Spring 2025